

UNIVERSIDAD DEL VALLE

Facultad de Ingeniería

Escuela de Ingeniería de Sistemas y Computación

Análisis de Algoritmos II

Informe de Proyecto

Proyecto de curso

El Problema de Minimizar la Polarización presente en una Población: MINPOL

Estudiantes: Juan Carlos Cruz Muñoz (1824389), David
Alejandro Enciso Gutierrez (2240581), Juan
Esteban Rodriguez Valencia (2042282),
Estiven Andres Martinez Granados (2179687)
Profesor: Jesús Alexander Aranda
Monitor: Samuel Galindo

Cali, Colombia

Julio de 2026

Índice

1. Introducción	2
2. Descripción del problema	2
2.1. Ejemplo del enunciado	2
3. Modelo matemático	3
3.1. Conjuntos y parámetros	3
3.2. Variables	3
3.3. Restricciones	3
3.4. Función objetivo	4
4. Justificación del modelo	4
5. Detalles importantes de implementación	5
5.1. Escalamiento entero	5
5.2. Conversión y validación	5
5.3. Interfaz gráfica	6
5.4. Portabilidad y decisión sobre Docker	6
6. Análisis de Branch and Bound	6
6.1. Ejemplo pequeño explicado paso a paso	6
6.2. Ejemplo de mayor búsqueda	8
7. Pruebas realizadas	9
8. Análisis de resultados	10
8.1. Variación de m	10
8.2. Variación del presupuesto	10
8.3. Consenso y costos extra	11
8.4. Bondades, falencias, eficiencia y optimalidad	12
9. Conclusiones	12
10. Video explicativo	13

1. Introducción

La polarización de una población puede interpretarse como el esfuerzo necesario para aproximar sus opiniones a un punto central. En este proyecto se estudia el problema MINPOL: dada una distribución inicial de personas entre opiniones numéricas y un presupuesto limitado para modificar dichas opiniones, además de un límite de movimientos, se deben seleccionar movimientos que minimicen la suma de distancias respecto de la mediana ponderada final.

Se construyó un modelo de programación lineal entera (PLE), no mixto ni no lineal: todas las variables de decisión son enteras o binarias y los productos entre variables se linealizan. El modelo se implementó en MiniZinc y se integró con una interfaz gráfica desarrollada en Python y Tkinter. La aplicación valida archivos de entrada, conserva exactamente los datos decimales, genera el archivo `DatosProyecto.dzn`, ejecuta el solver y presenta la matriz de movimientos, la distribución final, el costo, la cantidad de movimientos usados, la polarización y la mediana seleccionada.

2. Descripción del problema

Sea n el número de personas y m el número de opiniones posibles. La distribución inicial está dada por p_i , mientras que $v_i \in [0, 1]$ representa el valor de la opinión i . Mover una persona desde i hasta j tiene costo base $c_{i,j}$. Cuando la opinión de destino estaba vacía inicialmente, es decir $p_j = 0$, se suma el costo extra ce_j .

El costo de mover $x_{i,j}$ personas es:

$$x_{i,j} \left[c_{i,j} \left(1 + \frac{p_i}{n} \right) + \mathbf{1}_{p_j=0} ce_j \right].$$

El factor $1 + p_i/n$ hace que resulte más costoso intervenir grupos de origen grandes. El costo extra depende exclusivamente de la distribución inicial, no de si la opinión se ocupa posteriormente.

Además, mover una persona de la opinión i a la opinión j consume $|j - i|$ movimientos. La suma de esos movimientos no puede superar el parámetro $maxM$.

Si y_i es la cantidad final de personas con opinión i , la polarización se define como:

$$\text{Pol}(y, v) = \sum_{i=1}^m y_i |v_i - \text{mediana}(y, v)|.$$

2.1. Ejemplo del enunciado

El ejemplo tiene $n = 20$, $m = 5$, $p = (12, 4, 0, 4, 0)$, $v = (0, 0.25, 0.5, 0.75, 1)$, $ct = 20$ y $maxM = 18$. Tres salidas factibles son:

Movimientos	Costo	Movs.	Polarización	Distribución final
Ninguno	0	0	4	[12, 4, 0, 4, 0]
4 personas de 4 a 2	9.6	8	2	[12, 8, 0, 0, 0]
4 de 2 a 1; 2 de 4 a 1; 2 de 4 a 2	19.2	14	0.5	[18, 2, 0, 0, 0]

La tercera salida es óptima. MiniZinc certificó el valor 0.5 y una enumeración independiente comprobó las 447 distribuciones finales distintas alcanzables bajo presupuesto y límite de movimientos. Si el presupuesto aumenta a 24, se puede alcanzar consenso en la opinión 1 usando 16 movimientos, todavía dentro de $maxM = 18$.

3. Modelo matemático

La formulación se presenta mediante cinco componentes: conjuntos e índices, parámetros, variables de decisión, restricciones y función objetivo.

3.1. Conjuntos y parámetros

Sea $O = \{1, \dots, m\}$. Para $i, j, k \in O$:

- $n \in \mathbb{N}$: población total.
- $p_i \in \{0, \dots, n\}$: población inicial en la opinión i , con $\sum_i p_i = n$.
- $v_i \in [0, 1]$: valor de la opinión.
- $c_{i,j} \geq 0$: costo base por movimiento.
- $ce_j \geq 0$: costo extra del destino inicialmente vacío.
- $ct \geq 0$: presupuesto.
- $maxM \in \mathbb{N}^+$: máximo número de movimientos.

Se define el parámetro binario $e_j = 1$ si $p_j = 0$, y $e_j = 0$ en caso contrario. El costo unitario efectivo es:

$$a_{i,j} = c_{i,j} \left(1 + \frac{p_i}{n}\right) + e_j ce_j.$$

3.2. Variables

- $x_{i,j} \in \{0, \dots, n\}$: personas que pasan de i a j .
- $y_i \in \{0, \dots, n\}$: población final en i .
- $z_k \in \{0, 1\}$: selección de v_k como mediana.
- $q_{i,k} \in \{0, \dots, n\}$: linealización de $y_i z_k$.

3.3. Restricciones

No se modela como movimiento permanecer en la misma opinión:

$$x_{i,i} = 0 \quad \forall i \in O.$$

No pueden salir más personas de las existentes inicialmente:

$$\sum_{\substack{j \in O \\ j \neq i}} x_{i,j} \leq p_i \quad \forall i \in O.$$

La distribución final se obtiene mediante balance:

$$y_i = p_i - \sum_{\substack{j \in O \\ j \neq i}} x_{i,j} + \sum_{\substack{j \in O \\ j \neq i}} x_{j,i} \quad \forall i \in O,$$

$$\sum_{i \in O} y_i = n.$$

La restricción presupuestaria es:

$$\sum_{i \in O} \sum_{\substack{j \in O \\ j \neq i}} a_{i,j} x_{i,j} \leq ct.$$

La restricción de movimientos es:

$$\sum_{i \in O} \sum_{\substack{j \in O \\ j \neq i}} |j - i| x_{i,j} \leq maxM.$$

Para elegir exactamente una mediana:

$$\sum_{k \in O} z_k = 1.$$

El producto $q_{i,k} = y_i z_k$ se linealiza con:

$$\begin{aligned} q_{i,k} &\leq y_i, \\ q_{i,k} &\leq n z_k, \\ q_{i,k} &\geq y_i - n(1 - z_k), \\ q_{i,k} &\geq 0. \end{aligned} \quad \forall i, k \in O.$$

3.4. Función objetivo

$$\text{mín} \quad P = \sum_{i \in O} \sum_{k \in O} |v_i - v_k| q_{i,k}.$$

Para una distribución final fija, el valor que minimiza la suma de desviaciones absolutas es una mediana ponderada. Por eso la minimización conjunta sobre x y z selecciona automáticamente una mediana válida. En una población par puede existir un intervalo de medianas; alguno de sus extremos coincide con un v_k y conserva el mismo valor objetivo.

4. Justificación del modelo

Cada matriz entera x representa directamente los esfuerzos solicitados. La restricción de salida evita reutilizar personas que llegaron desde otra opinión y garantiza que cada individuo se mueva a lo sumo

una vez. Las ecuaciones de balance conservan la población. La restricción de presupuesto reproduce literalmente la fórmula del enunciado, incluido el costo extra por persona para destinos inicialmente vacíos. La restricción de movimientos reproduce el conteo $|j - i|$ exigido para cada persona movida.

La linealización de la mediana es exacta porque $0 \leq y_i \leq n$. Si $z_k = 0$, las restricciones fuerzan $q_{i,k} = 0$; si $z_k = 1$, fuerzan $q_{i,k} = y_i$. Por tanto, el objetivo es exactamente la polarización respecto del valor seleccionado.

En consecuencia, toda solución de MINPOL puede representarse en el modelo, y toda solución óptima del modelo proporciona movimientos válidos con la menor polarización posible.

5. Detalles importantes de implementación

5.1. Escalamiento entero

Los archivos MPL admiten decimales, pero el modelo utiliza enteros escalados para evitar errores de representación binaria. Si S_v y S_c son potencias de diez apropiadas:

$$V_i = S_v v_i, \quad C_{i,j} = S_c c_{i,j}, \quad CE_j = S_c ce_j, \quad CT = S_c ct.$$

El costo unitario se multiplica además por n :

$$U_{i,j} = C_{i,j}(n + p_i) + e_j CE_j n.$$

MiniZinc impone

$$\sum_{i,j:i \neq j} U_{i,j} x_{i,j} \leq CTn.$$

Esta transformación es algebraicamente equivalente al costo original. El conversor usa el tipo `Decimal` de Python y calcula automáticamente las escalas mínimas necesarias.

5.2. Conversión y validación

El módulo `mpl_parser.py` verifica:

- presencia de las $7 + m$ líneas requeridas;
- positividad de n y m ;
- dimensiones de todos los vectores y de la matriz;
- valores enteros no negativos en p y $\sum_i p_i = n$;
- $0 \leq v_i \leq 1$;
- costos y presupuesto no negativos y finitos;
- $maxM$ entero positivo.

Después genera `DatosProyecto.dzn`. La validación sigue el orden actualizado de la sección 3.1: n , m , p , v , ce , matriz c , ct y $MaxMvs$.

5.3. Interfaz gráfica

La interfaz utiliza Tkinter y permite cargar un MPL, inspeccionar los datos, generar el DZN y ejecutar `Proyecto.mzn`. La ejecución se realiza en un hilo separado para no bloquear la ventana. Los resultados se presentan en formato resumido y también se conserva la salida completa con estadísticas. El usuario puede elegir Chuffed, HiGHS o COIN-BC. Además, la vista de resultados recalcula y muestra siete comprobaciones: dominio entero no negativo, ausencia de automovimientos, disponibilidad por origen, balance y conservación de población, presupuesto, límite de movimientos y consistencia de la mediana con la polarización. Así, el cumplimiento de las restricciones puede inspeccionarse directamente sin interpretar la salida técnica.

5.4. Portabilidad y decisión sobre Docker

La aplicación usa únicamente la biblioteca estándar de Python para su interfaz y lógica de ejecución; MiniZinc es la única dependencia externa de producción. El entorno se comprueba con:

```
python3 ProyectoGUIFuentes/verificar_entorno.py
```

Este comando revisa Python, Tkinter, MiniZinc y los solvers compatibles. El `Readme.txt` incluye instrucciones para Windows, macOS y Linux y permite indicar una instalación no estándar mediante `MINIZINC_EXECUTABLE`.

Se evaluó Docker, pero no se adoptó como mecanismo principal: Tkinter requiere acceso al sistema gráfico y a los diálogos nativos. Exponer una GUI desde un contenedor requiere configuraciones diferentes para X11, Wayland y Docker Desktop. Un contenedor sería útil para pruebas automáticas sin interfaz, pero para el entregable gráfico la ejecución nativa es más directa y portable.

6. Análisis de Branch and Bound

Branch and Bound mantiene una solución entera incumbente que actúa como cota superior en un problema de minimización. La relajación de cada nodo proporciona una cota inferior. Un nodo se poda cuando es infactible, cuando su cota no puede mejorar al incumbente o cuando produce una solución entera. La optimalidad se demuestra cuando no quedan nodos cuya cota pueda mejorar la mejor solución.

6.1. Ejemplo pequeño explicado paso a paso

Se construyó la instancia reproducible `DatosProyecto/branch_and_bound/ejemplo_pequeno.mpl`, con $n = 4$, $m = 3$, $p = (2, 0, 2)$, $v = (0, 0.5, 1)$, $ce = (0, 0, 0)$, $ct = 3.5$, $maxM = 4$ y

$$C = \begin{pmatrix} 0 & 1 & 10 \\ 10 & 0 & 10 \\ 10 & 2 & 0 \end{pmatrix}.$$

Los únicos arcos no diagonales cuyo costo cabe en el presupuesto son $1 \rightarrow 2$ y $3 \rightarrow 2$. Sean $a = x_{1,2}$ y $b = x_{3,2}$. El costo efectivo de cada traslado por el primer arco es $1(1 + 2/4) = 3/2$, mientras que por

el segundo es $2(1 + 2/4) = 3$. La distribución resultante es $[2 - a, a + b, 2 - b]$. La opinión central, de valor 0.5, siempre es una mediana óptima para estas soluciones, por lo que la instancia se reduce a:

$$\begin{aligned} \text{mín} \quad & P = 2 - \frac{1}{2}(a + b) \\ \text{sujeto a} \quad & \frac{3}{2}a + 3b \leq \frac{7}{2}, \\ & 0 \leq a \leq 2, \quad 0 \leq b \leq 2, \\ & a, b \in \mathbb{Z}. \end{aligned}$$

En el nodo raíz se omite temporalmente la integralidad. La relajación lineal usa primero el traslado más barato y obtiene $(a, b) = (2, 1/6)$, con cota inferior $LB = 2 - \frac{1}{2}(2 + 1/6) = 11/12$. Como b es fraccionaria, Branch and Bound crea los dos subproblemas $b \leq \lfloor 1/6 \rfloor = 0$ y $b \geq \lceil 1/6 \rceil = 1$.

En la rama $b \leq 0$, la relajación produce $(a, b) = (2, 0)$, una solución entera con polarización 1. Esta solución se convierte en el incumbente y establece $UB = 1$. En la rama $b \geq 1$, la mejor solución relajada es $(a, b) = (1/3, 1)$, con $LB = 4/3$. Como es un problema de minimización y $4/3 \geq 1$, toda solución entera descendiente de esa rama es incapaz de mejorar el incumbente; por tanto, se poda por cota.

Nodo	Restricción añadida	Óptimo de la relajación	Decisión
Raíz	ninguna	$(2, 1/6)$, $LB = 11/12$	ramificar en b
Izquierdo	$b \leq 0$	$(2, 0)$, $P = 1$	entero; fijar $UB = 1$
Derecho	$b \geq 1$	$(1/3, 1)$, $LB = 4/3$	podar: $LB \geq UB$

Branch and Bound clásico — instancia MinPol pequeña

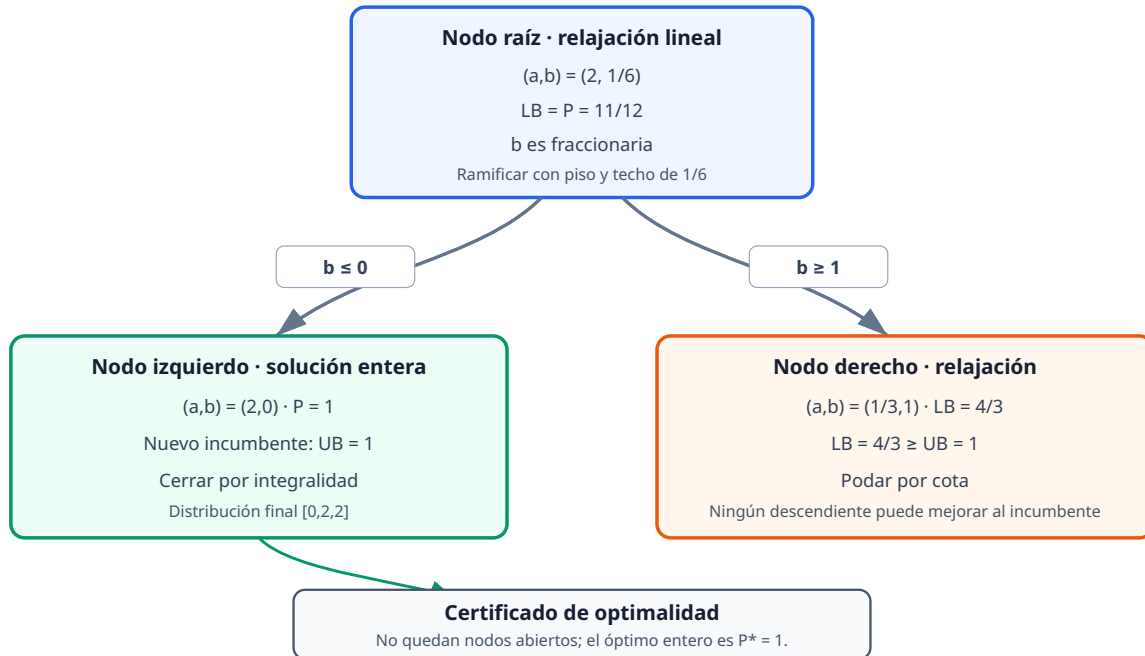


Figura 1: Árbol de Branch and Bound de la instancia pequeña: ramificación sobre una variable fraccionaria y poda por cota.

Al cerrar las dos ramas no quedan nodos abiertos y se certifica que $(a, b) = (2, 0)$ es óptima. En términos de MINPOL, se trasladan dos personas de la opinión 1 a la 2, con costo 3, dos movimientos, distribución final $[0, 2, 2]$, mediana seleccionada 0.5 y polarización 1. MiniZinc y HiGHS confirman ese óptimo con objetivo escalado 10. El solver industrial puede cerrar la instancia en el nodo raíz mediante preprocesamiento y cortes; el árbol anterior expone deliberadamente el procedimiento clásico sobre la relajación lineal para hacer visibles la ramificación y la poda.

6.2. Ejemplo de mayor búsqueda

En el ejemplo del enunciado, HiGHS resolvió el modelo en el nodo raíz: el preprocesamiento, la relajación y los cortes fueron suficientes para obtener y certificar el objetivo escalado 50. COIN-BC reportó también cero nodos adicionales. Chuffed mostró una búsqueda discreta más extensa:

Indicador	Valor
Nodos	225
Fallos	114
Reinicios	18
Profundidad máxima	9
Propagaciones	17 198
Backjumps	11
Objetivo escalado	50

Después de encontrar 50, Chuffed añadió implícitamente la exigencia de buscar una solución con objetivo menor. La salida final indica que la búsqueda con límite 49 no tiene solución, lo cual prueba el óptimo. Los árboles cambian entre solvers porque cada uno usa presolve, variables de ramificación, cortes, aprendizaje y reinicios diferentes. Por ello el número de nodos no debe compararse sin identificar solver, versión y configuración.

Para el modelo actualizado con $maxM$, la ejecución con Gecode 6.3.0 reportó 13 soluciones, 50 nodos, 11 fallos y profundidad máxima 18. Estas cifras se reproducen desde la raíz del proyecto con `minizinc --solver Gecode --statistics Proyecto.mzn DatosProyecto/ejemplos/ejemplo_pdf.dzn`.

La figura 2 muestra la ejecución interactiva del mismo modelo y DZN en Gecode Gist. Después de explorar todas las soluciones, el visualizador registró profundidad 19 (contando la raíz), 13 nodos solución, 17 nodos fallidos, 29 nodos de ramificación y cero nodos abiertos. El conteo visual no coincide directamente con las estadísticas del motor porque Gist clasifica y contabiliza los nodos dibujados, mientras el modo estadístico reporta eventos internos de búsqueda.

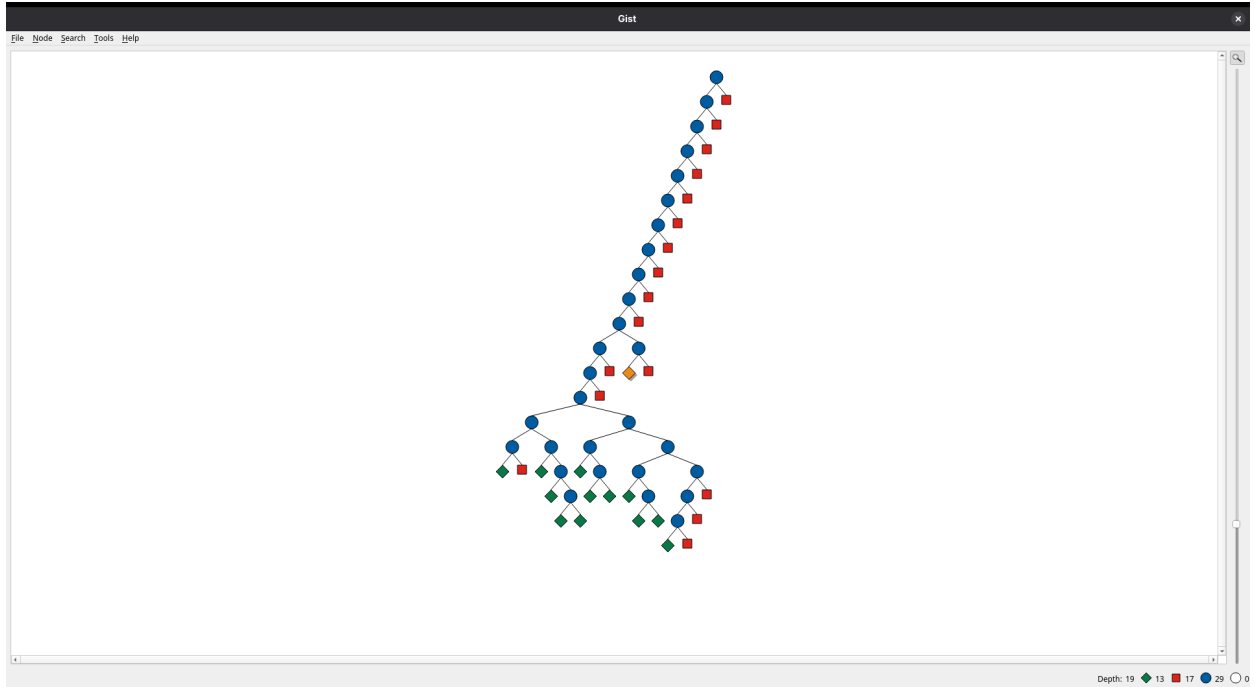


Figura 2: Árbol completamente explorado en Gecode Gist para el ejemplo del enunciado con la restricción $maxM$. La barra inferior confirma cero nodos abiertos.

7. Pruebas realizadas

La tabla principal se ejecutó con MiniZinc 2.9.7 y HiGHS 1.14.0. La batería de 30 entradas suministradas se resolvió con COIN-BC 2.10.13, que reproduce el óptimo de referencia 10.313 de MinPol15. Los tiempos corresponden al solver y son mediciones puntuales.

Instancia	n	m	ct	$maxM$	Movs.	Costo	Polarización	Tiempo (s)	Nodos
Ejemplo PDF	20	5	20	18	14	19.2	0.5	0.0255	1
Instancia 1	10	4	12	6	6	11.7	0	0.0121	1
Instancia 2	20	5	10	7	7	9.8	5.8	0.1048	9
Instancia 3	24	6	30	18	18	30	6.75	0.1867	11
Instancia 4	18	6	18.75	15	14	18.6667	2.75	0.2588	76
Instancia 5	40	8	45	25	24	44.85	11.7	0.3547	15

Todos los estados fueron óptimos: HiGHS reportó igualdad entre el objetivo incumbente y la cota. Además se ejecutaron pruebas unitarias del parser, el escalamiento, las validaciones, el procesamiento de resultados y la comprobación independiente de restricciones.

La batería completa se reproduce con `python3 Pruebas/ejecutar_pruebas.py`. Después, `python3 Pruebas/verificar_soluciones.py` recalcula cada solución sin confiar en las etiquetas impresas por MiniZinc: comprueba dominio, diagonal, disponibilidad, balance, conservación, costo, presupuesto, límite de movimientos y polarización mínima respecto de todos los valores candidatos a mediana.

También se procesaron las 30 entradas de `bateria_pruebas/mpl`:

Entradas	Cantidad	Resultado
MinPol1-30	30	Óptimo y verificaciones aprobadas

Las 30 entradas fueron convertidas a DZN antes de ejecutar el modelo con COIN-BC. En MinPol128 y MinPol29, $n = 125$ coincide con $\sum p_i = 125$ y permite reproducir los valores de referencia suministrados. La tabla completa se encuentra en `Pruebas/resultados_bateria_suministrada.csv` y se regenera con `python3 Pruebas/verificar_bateria_suministrada.py`.

Veintitrés valores coinciden textualmente con la tabla de referencia. Los siete restantes difieren únicamente en 0.001: el modelo entero escalado obtiene 9.687, 6.550, 14.172, 17.898, 23.567, 24.176 y 30.000. Esta diferencia es compatible con truncamiento de representaciones de punto flotante en los valores publicados.

8. Análisis de resultados

8.1. Variación de m

El modelo tiene dos familias principales de m^2 variables: movimientos x y productos auxiliares q . Su tamaño crece cuadráticamente. Sin embargo, los nodos no crecen de manera monótona: la instancia 4 con seis opiniones necesitó 76 nodos, mientras la instancia 5 con ocho necesitó 15. La simetría, los costos, la holgura y la cantidad de soluciones cercanas al óptimo también determinan la dificultad.

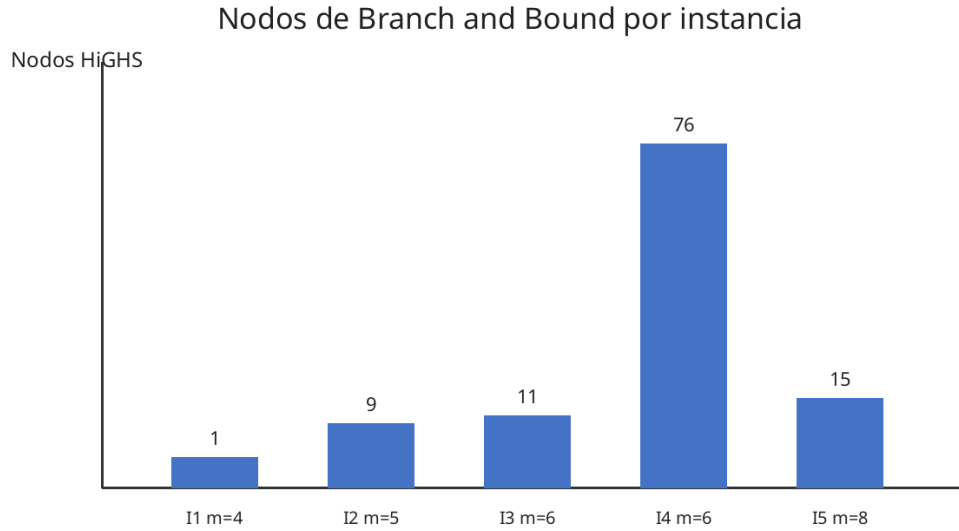


Figura 3: Nodos de Branch and Bound en las cinco instancias.

8.2. Variación del presupuesto

En el ejemplo, la polarización óptima pasa de 4 con $ct = 0$, a 2 con $ct = 10$, 0.5 con $ct = 20$ y 0 con $ct = 24$. Aumentar el presupuesto no puede empeorar el óptimo porque amplía el conjunto

factible. La mejora puede ser escalonada porque las personas son indivisibles. En esta comparación se mantiene $maxM = 18$, por lo que el cambio observado corresponde al presupuesto.

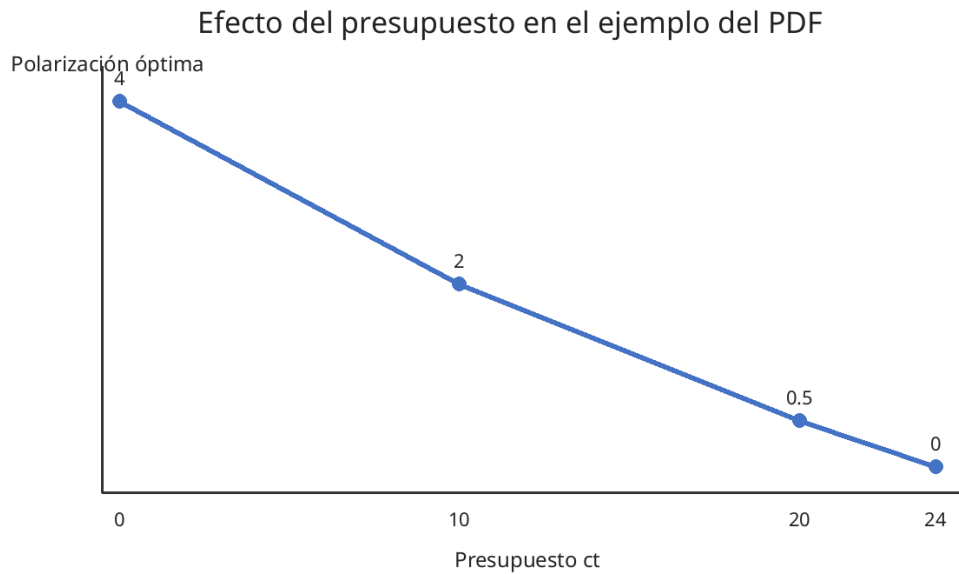


Figura 4: Presupuesto frente a polarización óptima.

8.3. Consenso y costos extra

La instancia 1 alcanza consenso con costo 11.7. El ejemplo requiere presupuesto 24 para lograrlo. En las demás configuraciones, concentrar toda la población en una opinión excede el presupuesto o el límite de movimientos.

La instancia 3 obtuvo polarización 6.75 cuando los destinos centrales vacíos tenían costo extra 12. Al fijar $ce = 0$, la polarización cayó a 6. La mejora es menor que en el modelo sin $maxM$, porque el límite de movimientos también restringe el uso de opiniones centrales. El costo extra puede cambiar tanto los movimientos como la mediana final.

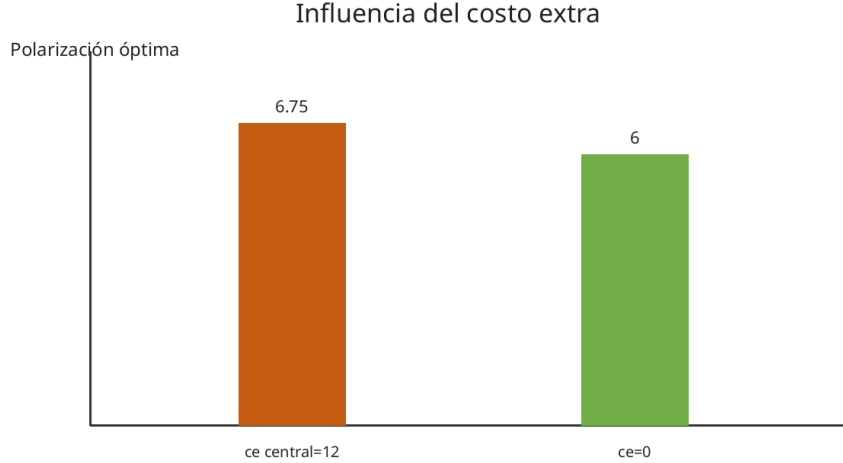


Figura 5: Resultado de la instancia 3 con y sin costo extra.

8.4. Bondades, falencias, eficiencia y optimalidad

Bondades. La formulación es genérica respecto de n y m , preserva exactamente los decimales mediante escalamiento entero y mantiene lineales tanto el presupuesto como la función objetivo. La linealización $q_{i,k} = y_i z_k$ permite seleccionar una mediana ponderada sin depender de una función no lineal ni de un solver específico.

Falencias. Las familias x y q contienen m^2 variables, y la linealización de q añade tres desigualdades por par (i, k) . Por ello, memoria y tiempo pueden crecer rápidamente cuando aumenta m . Además, el escalamiento con muchos decimales produce coeficientes enteros grandes y puede acercarse a los límites numéricos del backend. El modelo no rompe todas las simetrías que aparecen cuando varias opiniones tienen el mismo valor o costo.

Eficiencia. En las seis instancias resumidas en la tabla, HiGHS tardó entre aproximadamente 0.01 y 0.36 segundos y exploró entre 1 y 76 nodos. En las 30 entradas de la batería suministrada, COIN-BC empleó entre 0.0031 y 5.0107 segundos y exploró entre 0 y 184 nodos, con $3 \leq m \leq 20$ y $5 \leq n \leq 200$. No se observa crecimiento monótono con m : la estructura de costos, la simetría y la holgura del presupuesto influyen además del tamaño cuadrático del modelo. Las validaciones previas y el uso de coeficientes enteros evitan enviar al solver entradas inválidas o aproximadas.

Optimalidad. El modelo no usa una heurística para decidir el valor final: los solvers completan una búsqueda exacta. En HiGHS, la igualdad entre objetivo primal y cota dual y una brecha de 0 % certifica el óptimo. En Chuffed, después del incumbente se demuestra que no existe una solución con objetivo entero escalado estrictamente menor. Estas certificaciones, junto con las pruebas sobre varias instancias, sustentan la optimalidad de las soluciones informadas.

9. Conclusiones

1. El problema MINPOL puede representarse exactamente mediante un modelo entero lineal al seleccionar la mediana con variables binarias y linealizar los productos correspondientes.

2. El escalamiento entero conserva los decimales de entrada y evita depender de tolerancias de punto flotante en la formulación.
3. El presupuesto controla directamente la reducción posible de la polarización; el consenso aparece únicamente cuando existe una concentración completa cuyo costo no excede ct y cuyos movimientos no exceden $maxM$.
4. Los costos extra penalizan el uso de opiniones inicialmente vacías y pueden impedir que las opiniones centrales, aunque cercanas a la mediana, formen parte de la solución.
5. El número de opiniones incrementa el tamaño cuadráticamente, pero la dificultad efectiva depende también de la estructura de costos y de la simetría.
6. Las diez pruebas de integración, las ocho pruebas unitarias y las 30 entradas válidas de la batería suministrada respaldan la corrección del modelo, el conversor y la comprobación independiente de restricciones.

10. Video explicativo

PENDIENTE: reemplazar este texto por el enlace público o institucional al video de máximo 15 minutos antes de entregar.